Detailed Summary of the v4 ETH Report

Certified Symmetry, Degree-Aware Precision,
Exact Counts, Runtime, and Memory

Dr. Muralidhara Subbarao
Technical Report Prepared with the Assistance of AI Models
https://www.integralresearch.net

October 27, 2026

Abstract

This report summarizes revision 4 of the Ram Calculus investigation of OpenAI's manuscript *The Exponential Time Hypothesis is False*. The principal advance over revision 3 is implementation and measurement: certified orbit aggregation is tested inside an exact finite CNF pipeline, and degree-aware precision is tested in actual modular contraction and interpolation. All 920 recorded worker outputs validate across 11 CNF cases and eight weighted-contraction cases. On the eight cases with a strict certified orbit-call reduction, orbit aggregation has median speedups of 1.11–4.95 over the already aggregated method. Its largest median speedup over the original subset-pair method is 37.50, but peak traced allocation rises rather than falls. Smaller precision budgets recover the same exact values; the isolated three-bit reduction does not give a consistent runtime or appreciable memory advantage. A stronger two-bit decoder is proved only for a specified one-edge family. We explain the mathematics, benchmark boundaries, failures, and requirements for a full-algorithm improvement. Neither an improved unrestricted SAT exponent nor an independent validation of the source's complete ETH claim is established. This is a synthesis of the recorded v4 evidence, not a new benchmark run or a new literature-status audit.

Contents

1 Main conclusions and scope **3**

1.1 What v4 adds to the earlier reports . . . 3

2 What was tested and what agreement means **4**

2.1 Two distinct experiments . . . 4

2.2 Protocol and sample size . . . 4

2.3 Correctness and memory contracts . . . 4

3 Why the symmetry reduction is exact **5**

3.1 What the certificate checks . . . 5

3.2 Fixtures and exact counts . . . 6

4 Demonstrated symmetry speedups and their costs **7**

4.1 What is a genuine incremental gain . . . 7

4.2 Memory increases, including when symmetry is absent . . . 7

4.3 Failures to accelerate and the direct-control warning . . . 8

5 The degree-aware precision results **8**

5.1 The target and the information actually available . . . 8

5.2 Why knowledge of degree saves bits . . . 9

5.3 What each precision comparison isolates . . . 10

5.4 The stronger two-bit result and its exact limitation . . . 10

6 Measured precision runtime and memory **10**

6.1 The one-edge case is already a direct-summation leaf . . . 12

7 Failure controls and the claims they exclude **12**

8 What is attributable to Ram Calculus **13**

9 What remains necessary for a full-algorithm improvement **13**

9.1 Implement the active recursive regime . . . 13

9.2 Certify degree, target, and precision at every node . . . 14

9.3 Make symmetry survive the source's constructions . . . 14

9.4 Measure realistic performance and establish a complete cost bound . . . 14

9.5 Evidence-to-goal checklist . . . 15

10 Conclusion and supporting files **15**

1 Main conclusions and scope

The v4 report [1] makes an important but carefully bounded advance: two ideas previously supported by identities and small exact checks are implemented in matched local computations. Its strongest observed result is an additional speedup from *certified symmetry*, not a universal speedup from reducing the number of precision bits. These conclusions must be separated:

1. **Exactness was retained on every tested output.** The counting methods agree on all CNF counts and complete recovered coefficient vectors. The precision methods agree on the specified integer contraction outputs and on the complete local polynomials modulo their working moduli. This is more demanding than checking a few final scalar values.
2. **Symmetry can materially reduce measured work on structured instances.** The additional orbit method outperforms aggregation in all eight cases where certification strictly reduces its call count. It does not make arbitrary instances symmetric or solve them in polynomial time.
3. **This symmetry implementation spends memory to save work.** Its catalogue raises peak traced allocation to 1.40–1.93 times the aggregation baseline across the suite. Fewer calls do not imply smaller workspace.
4. **The precision theorems are stronger than the measured performance conclusion.** The smaller budgets are valid exact-recovery guarantees, but the three-bit saving alone has no consistent runtime or appreciable traced-memory benefit in these measurements.
5. **The full algorithm has not been benchmarked.** There is no measured end-to-end speedup for the source's complete recursion, no new general counting exponent, and no independent verification of every dependency of the source's headline result.

The source examined in v4 is the manuscript revised October 25, 2026, with local source pinned to commit

`42907c9e999cc00d6abe7a50bfc1023250edd1b9`.

The benchmark's word "original" means a local implementation of a specified routine in that source, not a timing of an OpenAI-supplied executable. The conclusions concern that pinned object. They should not be read as a statement about subsequent repository changes or community acceptance of the claimed refutation of ETH.

1.1 What v4 adds to the earlier reports

Revision 2 measured the original subset-pair method against aggregated Boolean jets. Revision 3 added target-identifiability conditions, sharp precision losses on stated polynomial classes, and an orbit-aggregation identity motivated by the expanded Ram Calculus corpus. Revision 4 adds three concrete contributions:

- a sufficient, executable symmetry certificate for an entire finite deformation layout, followed by orbit reduction inside the same exact recovery pipeline;
- actual modular evaluations and interpolation at smaller certified precisions, with ablations separating decoder choice from arithmetic width;

- a sharper two-bit target formula, together with its limitation and lower-bound argument, for the precisely specified one-edge family used in the precision experiment.

The older results retained in v4 are historical evidence, not additional repetitions of the new benchmark. The new contribution is not a claim of historical priority for group actions, Hermite interpolation, or two-adic divisibility. It is the particular derivation, certification, implementation, and empirical assessment recorded in this investigation.

2 What was tested and what agreement means

2.1 Two distinct experiments

The **symmetry experiment** concerns the source's Section-3 subset-pair routine. Its methods are O (original pair loop), A (aggregated Boolean jets), G (certified orbit aggregation), and a direct truth-table counting control. O, A, and G return the same full deformation polynomial as well as its target count. They use the same exact arithmetic, restricted-count kernel, interpolation nodes, truncation order, and full Hermite solve. G is not advantaged by returning less information.

The **precision experiment** concerns a local Appendix-D Cauchy deformation and tapered-Hermite recovery calculation. Five modular variants and a direct integer-contraction control are compared. These outputs are weighted integer contraction coefficients, *not CNF solution counts*. The two experiments are not successive stages of a new complete solver. Their speedup ratios cannot be multiplied to obtain a predicted end-to-end improvement.

2.2 Protocol and sample size

The new runs used Python 3.12.14 on Windows 11, an Intel Family 6 Model 140 processor, and eight logical CPUs. Workers were launched sequentially, with seven timing repetitions per case and method in seeded, randomized interleaved order. Two separate workers measured memory, and one further worker collected diagnostics. There was no CPU-affinity or frequency isolation. The recorded sample counts are

$$11 \cdot 4 + 8 \cdot 6 = 92 \quad \text{case–method combinations,}$$

$$92 \cdot 7 = 644 \text{ timed}, \qquad 92 \cdot 2 = 184 \text{ memory}, \qquad 92 \cdot 1 = 92 \text{ diagnostic executions.}$$

Thus 920 validated worker outputs represent repeated measurements on *19 fixtures*, not 920 independent mathematical instances. Of these outputs, 730 belong to full-polynomial methods and 190 to direct target-only controls. Separate selftests and an early pilot are not included in these timing statistics.

Elapsed time includes preparation, symmetry certification, orbit catalogue construction, arithmetic, and recovery. It excludes process startup, imports, input deserialization, output serialization, and independent oracle validation. Reported speedups are ratios of medians. The seven samples have substantial variation in several cases, so these ratios are descriptive observations rather than confidence intervals or universal performance guarantees.

2.3 Correctness and memory contracts

For the CNF methods the checks cover the exact truth count, every jet coefficient, every recovered polynomial coefficient, and required zero padding. The oracle independently enumerates assignments

and convolves coefficients, but shares the specified layout constructor. Consequently, the layout proof and certificate remain necessary: the oracle is not a wholly independent implementation of every input-construction step.

For the modular methods, independent rational-polynomial calculations check both complete local cubics at the actual modulus and every retained integer target coefficient. Extension-field coordinates that should vanish are also checked. Direct controls are compared on their target outputs only; they do not generate the same intermediate polynomials.

Memory is the median of two peak `tracemalloc` measurements, reported in KiB (1 KiB = 1024 bytes). These peaks include preparation and returned arrays, but exclude already-deserialized inputs and do not account for every native allocation. Raw Windows working-set peaks also exist, but include interpreter/import history and tracer overhead. Neither measure is a theorem about asymptotic workspace or an isolated measure of arithmetic storage.

3 Why the symmetry reduction is exact

Let n be the number of Boolean variables, q the correction order, Q the number of interpolation nodes, and $r = \min(q, n)$. The original routine visits a subset and an internal choice, giving

$$N_{\mathrm{O}} = Q \sum_{j=0}^{r} 2^j \binom{n}{j}.$$

Aggregation encodes the internal choices in an enlarged coefficient algebra and reduces this to

$$N_{\mathrm{A}} = Q \sum_{j=0}^{r} \binom{n}{j}.$$

The reduced number of calls is not a free factor-of-2^q speedup: the aggregate call may manipulate a larger mixed-jet state.

The next reduction is available when a permutation group G preserves the complete deformed family. If C_U is the coefficient block produced for subset U, the certificate establishes

$$C_{gU} = C_U \qquad (g \in G).$$

Writing $\mathcal{O}_q$ for its orbits on the retained subsets gives

$$\sum_{|U| \le r} C_U = \sum_{\mathcal{O} \in \mathcal{O}_q} |\mathcal{O}| C_{U_{\mathcal{O}}}, \qquad N_{\mathrm{G}} = Q|\mathcal{O}_q|.$$

The proof is a bijective change of Boolean assignment under the permutation, followed by grouping equal summands. It preserves the full coefficient block, not only its constant coefficient. Orbit sizes are integer multiplicities; there is no division by a group order that might be noninvertible in the coefficient algebra.

3.1 What the certificate checks

The implementation checks the symbolic multiset of signed factors, literal multiplicities, exact deformation-row coefficients, fixed boundary measurements, group membership, and every interpolation node's assignment of factors to groups. Candidate generators are adjacent swaps inside a

group's nonboundary coordinates. This is a conservative sufficient certificate, not a complete search for all automorphisms.

Symmetry of the CNF alone is insufficient: a deformation row or a node-dependent group assignment can break it. Equal Hamming weight is also insufficient. For example, $(1, 0)$ and $(0, 1)$ have equal weight but are distinguished by multiplication by the first labelled variable. The quotient must be closed under the actual operations and must preserve the requested target. A smaller state description without these conditions can give a wrong count.

3.2 Fixtures and exact counts

The cases include empty and unsatisfiable formulas, symmetric formulas, partially broken and completely broken deformation symmetry, and node-varying factor assignments. Their small sizes and special structures make them correctness and local-performance fixtures, not a hard-SAT benchmark distribution.

In one family, $n = 2(k + 1)$, there are two groups with a boundary variable in each, and each nonboundary variable in one group is joined by an OR clause to the other group's boundary variable. The count is $(2^k + 1)^2$. This gives an independent closed-form check and also exposes how easy the underlying counting problem is.

S11 is more demanding for the deformation interface. Its four clauses are $X_i \vee X_j$ for $i \in \{1, 2\}$ and $j \in \{4, 5\}$, with the other two variables free. Hence its count is $4(4 + 4 - 1) = 28$. Rational deformation rows and factor assignments change between interpolation nodes, yet two internal swaps remain certified. Its 57 retained subsets form 31 orbits, so two nodes require 62 rather than 114 aggregated calls. This verifies that the implementation is not limited to a fixed factor assignment.

Table 1: Exact CNF counts and restricted calls across all nodes. Gen. is the number of accepted generators, not the order of the generated group. All three methods agree on the count and full recovered coefficients.

ID	n	q	Count	Gen.	O calls	A calls	G calls
S01	6	0	64	2	2	2	2
S02	6	2	0	2	292	88	52
S03	6	2	25	2	292	88	52
S04	8	2	81	4	645	185	65
S05	8	3	81	4	2308	372	100
S06	10	2	289	6	1206	336	78
S07	8	4	81	4	5091	489	117
S08	8	3	81	3	2308	372	148
S09	6	2	25	0	292	88	88
S10	6	2	40	0	146	44	44
S11	6	4	28	2	946	114	62

4 Demonstrated symmetry speedups and their costs

Table 2: Median milliseconds for the full finite pipeline. O/G and A/G greater than one favor G. Direct is a target-only control with a smaller output contract.

ID	O ms	A ms	G ms	O/G	A/G	Direct ms
S01	0.486	0.592	0.625	0.78	0.95	0.047
S02	164.518	55.744	50.041	3.29	1.11	0.260
S03	231.618	95.140	70.163	3.30	1.36	0.401
S04	1,168.400	251.432	104.235	11.21	2.41	1.564
S05	3,609.031	594.488	120.167	30.03	4.95	1.716
S06	5,744.987	1,527.460	524.953	10.94	2.91	7.170
S07	6,960.293	453.487	185.596	37.50	2.44	1.884
S08	2,578.626	655.570	211.188	12.21	3.10	1.691
S09	217.112	75.735	65.543	3.31	1.16	0.322
S10	136.587	72.493	73.123	1.87	0.99	0.303
S11	1,398.999	378.203	256.360	5.46	1.48	0.388

4.1 What is a genuine incremental gain

Across all 11 cases, G has a lower median than A in nine and than O in ten. The eight cases with a strict certified call reduction have A/G speedup ratios of 1.11–4.95. The remaining apparent gain, S09, has no accepted symmetry generator and no call reduction; its lower G median must not be credited to orbit compression.

Two examples distinguish the new benefit from the pre-existing aggregation benefit:

- **S05:** O takes 3609.031 ms, A takes 594.488 ms, and G takes 120.167 ms. The new G-over-A gain is 4.95, while the combined improvement over O is 30.03. Calls decrease from 2308 to 372 to 100.
- **S07:** the medians are 6960.293, 453.487, and 185.596 ms. The gain over O is 37.50, but the incremental gain over A is 2.44. Calls decrease from 5091 to 489 to 117. The headline 37.50 must not be attributed entirely to the newly added symmetry layer.

For S05, G's observed range is 106.305–354.555 ms, versus 367.473–1180.226 ms for A. For S07, the ranges are 114.646–314.599 and 411.937–1078.258 ms. These provide useful evidence on those fixtures, without establishing a distribution-wide or worst-case guarantee. Other cases have overlapping timing ranges.

4.2 Memory increases, including when symmetry is absent

Table 3: Median peak traced Python allocations in KiB. G/A greater than one means the orbit implementation used more traced memory than aggregation.

ID	O KiB	A KiB	G KiB	G/A	Direct KiB
S01	4.359	4.492	8.664	1.93	1.039

ID	O KiB	A KiB	G KiB	G/A	Direct KiB
S02	27.937	27.897	41.788	1.50	2.312
S03	26.241	26.202	38.108	1.45	2.312
S04	39.065	38.815	55.925	1.44	2.312
S05	42.737	42.659	59.862	1.40	2.312
S06	57.026	56.651	79.956	1.41	2.344
S07	37.640	37.835	54.960	1.45	2.312
S08	42.733	42.655	59.921	1.40	2.312
S09	26.241	26.202	38.624	1.47	2.312
S10	12.006	12.271	20.541	1.67	2.312
S11	21.990	22.623	34.205	1.51	2.312

G's traced allocation is higher than A's in every case, by a factor of 1.40–1.93. For S05 it rises from 42.659 to 59.862 KiB; for S07 it rises from 37.835 to 54.960 KiB. The implemented G method materializes a subset set and orbit catalogue, whereas the baselines stream subsets. It also coexists with mixed-jet state, recovery matrices, and returned coefficient arrays. Which object determines the peak depends on the case.

The catalogue construction was corrected before the main timings to use a single traversal rather than repeated minimum-set scans. This removes an avoidable quadratic scanning pattern on singleton orbits, but it does not turn the catalogue into a constant-memory representation. A streaming or compact orbit representation remains a potential engineering improvement, not an outcome demonstrated by these runs.

4.3 Failures to accelerate and the direct-control warning

S01 has $q = 0$, so all three methods already make two calls. G's median 0.625 ms is slower than A's 0.592 ms and O's 0.486 ms. S10 has no accepted generator and G's 73.123 ms is slightly slower than A's 72.493 ms. Certification and catalogue overhead are real even when no work can be eliminated.

Direct truth-table counting is faster than G on all 11 fixtures. For example, it takes 1.884 ms on S07, compared with G's 185.596 ms. This is not an equal-output comparison because Direct does not return the full deformation polynomial. Nevertheless, if the practical objective is *only the count* on these small inputs, Direct is the faster tested way to obtain it. The local gains over O do not establish superiority over direct counting, competitive counting software, or the source's best complete path on arbitrary inputs.

5 The degree-aware precision results

5.1 The target and the information actually available

The precision calculation uses the source's one-edge local Cauchy entry

$$W(T) = -\frac{(T+2)(T+1)(T-1)}{2} = 1 + \frac{T}{2} - T^2 - \frac{T^3}{2}.$$

For nonnegative integer-coefficient entries A_0, A_1, B_0, B_1 in $\mathbb{Z}[V]/(V^\ell)$, the actual evaluated polynomial is

$$F(T) = \sum_{x,y\in\{0,1\}} A_y B_x \, l_y(W(T)x), \qquad l_0(z) = 1 - z, \quad l_1(z) = z.$$

Its target is the truncated product sum $F(0) = A_0B_0 + A_1B_1$. The local parameters are $a = u = M = 1$, $L = 2$, and dilation $\alpha = 4$.

At both $t = -1$ and $t = 1$, each modular variant evaluates $F(t + 4\zeta)$ at $\zeta = 0, 1, U, 1 + U$ in

$$(\mathbb{Z}/2^b\mathbb{Z})[U, V]/(U^4 + U + 1, V^\ell).$$

Pairwise differences of these points are units: $U^4 + U + 1$ is irreducible modulo two. Two ordinary Lagrange interpolations recover the complete local cubics. Thus every modular variant performs the same eight contraction evaluations, 32 Boolean summands, and two full cubic interpolations. The precision change does not reduce that evaluation count.

The measured jets are $h_{t,j} = [z^j]F(t + 4z)$. They are integral even though $F(T)$ can have half-integral coefficients. Contents of integer Cauchy factors are canceled before modular evaluation; two is never inverted as a unit modulo 2^b. The desired output modulus 2^p exceeds

$$\ell\big(\max A_0 \max B_0 + \max A_1 \max B_1\big),$$

where each maximum is over the corresponding coefficient array. This ensures that the nonnegative target coefficients are recovered as *exact integers*, not only as residues. Terms of degree at least ℓ in V are outside the declared truncated algebra.

5.2 Why knowledge of degree saves bits

For the six channels $j = 0, 1, 2$ at $t = -1, 1$, the following exact formulas hold on their respective polynomial classes:

$$\deg F \le 5: \quad F(0) = \frac{64(h_{-1,0} + h_{1,0}) + 10(h_{-1,1} - h_{1,1}) + h_{-1,2} + h_{1,2}}{128}, \tag{1}$$

$$\deg F \le 3: \quad F(0) = \frac{8(h_{-1,0} + h_{1,0}) + h_{-1,1} - h_{1,1}}{16}. \tag{2}$$

They follow by checking monomials and linearity. If each jet has an unknown error divisible by 2^b, a denominator 2^δ costs at most δ output bits. The first formula therefore suffices at $b = p + 7$; the second suffices at $b = p + 4$. Integer numerators are divided exactly before reduction modulo 2^p.

The cubic bound is certified for this local construction, not guessed from small numerically observed coefficients. Revision 3 proves that the common losses 1, 4, and 7 are optimal on its stated full degree-one, degree-three, and degree-five classes for these six residue channels. For example, the witness $(T + 1)(T - 1)^2$ has odd target at zero and all six scaled jets divisible by 16; scaling it by 2^{p-1} shows that one fewer common bit cannot suffice for the full cubic data class. These are information bounds for a specified observation contract, not lower bounds for every counting algorithm or every realizable tensor family.

The broader principle is target recovery rather than full coefficient recovery. With coefficient vector c, measurements $h = Ac$, and target ℓc, exact recovery is possible exactly when $\ker A \subseteq \ker \ell$, equivalently when $A^T\lambda = \ell^T$ has a solution. Choosing a valid target formula can improve its denominator valuations. This does not itself prove that obtaining its measurements is cheaper.

5.3 What each precision comparison isolates

Method	Working modulus	Decoder and interpretation
S64	2^{p+64}	Source allowance; six-channel formula
C7	2^{p+7}	Same six-channel formula; sharper certificate
D7	2^{p+7}	Cubic decoder; unchanged bit budget
D4	2^{p+4}	Same cubic decoder; three fewer bits
R2	2^{p+2}	Additional one-edge structural certificate

S64's 64 extra bits are a sufficient conservative allowance, not a proved minimum required by the source. C7 already exploits the earlier target-specific certificate. D7 versus D4 is the clean comparison for the *additional three-bit degree-aware saving*. D4 versus S64 combines both improvements and must not be described as testing degree awareness alone. The source's fixed Hermite weights are compiled in advance and checked against its literal truncated-reciprocal formula, so the reference is not charged repeated symbolic work unnecessarily.

5.4 The stronger two-bit result and its exact limitation

The actual one-edge family is smaller than the full cubic space:

$$F(T) = H + CW(T), \qquad H = A_0(B_0 + B_1), \quad C = (A_1 - A_0)B_1.$$

Because $W(-1) = 0$, $W(0) = 1$, and $[z]W(-1 + 4z) = 4$, its target satisfies

$$F(0) = h_{-1,0} + \frac{1}{4}h_{-1,1}. \tag{3}$$

This is R2. It recovers $H + C$ from H and $4C$, so $p + 2$ working bits suffice coefficientwise. Two extra bits are necessary if only the declared six-channel residues are supplied: the zero function and $2^{p-1}W(T)$ agree in those residues modulo 2^{p+1} but have different targets modulo 2^p. The latter polynomial is realized by $A_0 = B_0 = 0$, $A_1 = 1$, $B_1 = 2^{p-1}$.

There is no contradiction with the four-bit bound for all cubics, because R2 uses the additional restriction $F \in \operatorname{span}\{1, W\}$. For $F(T) = T^2$, equation (3) gives $1 - 8/4 = -1$ rather than zero. Degree three alone does not certify R2. Nor is the two-bit lower bound a restriction on an algorithm given the original tensors, which can compute their contraction directly.

R2 retains the same eight evaluations and two full local interpolations in the benchmark. Any further implementation that omits some evaluations would be a different workload requiring its own recovery proof and timing comparison; v4 does not claim that result.

6 Measured precision runtime and memory

Table 4: Precision cases and exact output sizes. Input bits are the entry creation precision; p is the certified target precision. Each method agrees on all retained coefficients.

ID	Input bits	ℓ	p	Output coefficient bits
P01	8	1	17	17–17

ID	Input bits	ℓ	p	Output coefficient bits
P02	64	1	129	129–129
P03	512	1	1025	1025–1025
P04	2048	1	4096	4096–4096
P05	8	4	19	17–19
P06	64	4	131	129–131
P07	8	16	21	17–21
P08	64	16	133	129–133

All eight cases agree exactly under every method. The scalar example P01 returns 75,637. P04 returns a 4096-bit integer; the nilpotent cases return all ℓ retained coefficients. The complete values are preserved in the raw JSON records, not replaced by floating-point approximations.

Table 5: Median milliseconds for precision variants. All non-direct methods retain eight evaluations and two complete cubic interpolations.

ID	S64	C7	D7	D4	R2	Direct
P01	3.480	3.078	2.599	2.228	2.302	0.081
P02	3.739	2.883	3.427	3.724	3.543	0.088
P03	5.146	5.755	5.546	5.193	5.109	0.091
P04	19.963	26.865	25.878	23.571	24.848	0.129
P05	7.972	5.800	8.655	8.917	9.020	0.107
P06	4.725	5.071	4.601	5.524	4.297	0.075
P07	51.089	66.823	49.844	40.387	57.809	0.206
P08	68.413	69.285	74.091	61.415	59.586	0.274

D4 has a lower median than S64 in four of eight cases; S64/D4 ranges from 0.85 to 1.56. It is therefore incorrect to summarize the precision experiment as a uniform speedup. For instance, P01 improves from 3.480 to 2.228 ms, but P04 changes from 19.963 to 23.571 ms and P06 from 4.725 to 5.524 ms.

For the isolated D7–D4 comparison, D4 has a lower median in five cases and a higher median in three. D7/D4 ranges from 0.83 to 1.23. The restricted R2 method is also not uniformly faster: D4/R2 ranges from 0.70 to 1.29. In P07, for example, R2 takes 57.809 ms versus D4's 40.387 ms despite using two fewer bits. Arithmetic-width savings, unit-lifting schedules, decoder overhead, allocation granularity, and timing variability can all matter; these runs do not identify a unique cause of each reversal.

The observed ranges reinforce that restraint. In P01, D7 spans 1.675–3.545 ms and D4 spans 1.771–6.823 ms. In P08, D7 spans 38.580–153.141 ms and D4 spans 37.400–108.107 ms. There is no statistical-confidence claim that a small median difference will recur on another machine or implementation.

Table 6: Median peak traced Python allocation in KiB for the precision experiment.

ID	S64	C7	D7	D4	R2	Direct
P01	56.610	54.677	54.677	54.677	54.677	2.125
P02	58.216	57.880	57.880	57.880	57.860	2.188

ID	S64	C7	D7	D4	R2	Direct
P03	67.102	66.633	66.633	66.637	66.641	2.656
P04	100.004	99.457	99.457	99.434	99.426	4.250
P05	130.188	128.028	128.028	128.028	128.028	2.609
P06	132.544	131.927	131.927	131.942	131.985	2.812
P07	211.383	208.539	208.539	208.539	208.539	5.703
P08	220.609	216.562	216.562	216.566	216.578	6.469

Across the precision suite, the D4/S64 traced-peak ratios round to 0.97–1.00. For P01 the change is 56.610 to 54.677 KiB, approximately 3.4 percent; for P04 it is 100.004 to 99.434 KiB, approximately 0.6 percent. These are modest whole-pipeline allocation savings, not a reduction proportional to the 60-bit difference between the budgets. The D4/D7 ratios round to 1.00 in every case; small differences can be seen in the unrounded records and table. The three-bit change yields no appreciable traced-memory reduction.

6.1 The one-edge case is already a direct-summation leaf

Direct integer contraction is faster than D4 in all eight cases and has much smaller measured allocation peaks. For P08, Direct takes 0.274 ms and 6.469 KiB, versus D4's 61.415 ms and 216.566 KiB. It returns the same target coefficients, but not the deformed local polynomials.

More importantly, the source's complete algorithm already directly sums sufficiently small networks. The one-edge fixture does not meet its large-size active packet conditions; the benchmark deliberately forces the local deformation to measure that arithmetic component. Thus even a positive D4-over-S64 result is *not* a gain over the complete source algorithm's best path on that input. V4 establishes correctness of smaller budgets in an actual arithmetic implementation, not a faster general recursive tensor-contraction algorithm.

7 Failure controls and the claims they exclude

The negative tests are essential to interpreting the positive results.

1. **Unchecked symmetry can change the answer drastically.** Bypassing the certificate on a recorded asymmetric fixture changes its count from 25 to 2,333,861,705 and corrupts the recovered coefficients. The tests also reject a nonpermutation and reject invalid generator candidates for fully asymmetric deformation rows.

2. **A degree certificate cannot be replaced by optimism.** Applying the cubic target formula to T^4 returns -1 rather than zero; applying the six-channel degree-five formula to T^6 returns 1. R2 fails already on T^2 unless its stronger structural assumption holds. These are exact algebraic failures, not rounding errors.

3. **A good final count can hide a wrong polynomial.** The precision tests deliberately corrupt a nonconstant cubic coefficient that a target-only comparison could miss. Full coefficient validation detects it. Symmetry is likewise checked on mixed coefficients, not only on one recovered number.

4. **Increasing a modulus is not permission to change an entry.** An entry created and stored as $(1 + 1) \bmod 2 = 0$ must retain the chosen integer lift zero. Re-evaluating its original

expression as two at a larger modulus changes the input. The precision calculations keep creation-time representatives fixed.

5. **Modular correctness is not automatically integer correctness.** Insufficient output range can leave several integers compatible with a residue. The explicit nonnegative range bound is therefore part of the certificate. Nonunit inversion and insufficient precision are also rejected rather than silently approximated.

The symmetry selftest checks six layouts under all three non-direct methods and 176 mixed-coefficient generator comparisons. The precision selftests check all eight cases under all five modular methods, direct controls, cardinal interpolation identities, and agreement with the literal source Hermite formula. These checks strengthen the local implementation evidence; they do not certify every source theorem or every possible user-created layout.

8 What is attributable to Ram Calculus

The relevant Ram Calculus contribution is an organizing framework for finite moments, jets, target recovery, exact quotient closure, and certified inversion. It directs attention to questions that are easy to miss in a formal change of representation: what information the target actually requires, whether a proposed coarse state is closed under updates, and which denominator valuations control exact recovery.

In this investigation, that framework leads to usable identities, explicit certificates, negative tests, and matched implementations. The 13-volume corpus contribution in v3 supplied a crosswalk to target identifiability, degree-aware recovery, and symmetry/closure ideas; v4 supplies measurements of selected implementations and the narrower R2 refinement. It does not re-audit every result in those volumes.

The resulting mechanisms can also be expressed in conventional terms: Boolean generating functions, finite group actions, rational linear algebra, Hermite interpolation, and two-adic divisibility. An equivalent conventional implementation is not proved slower, less accurate, or less capable. Every correct method here produces the same exact answer, so there is no approximate-accuracy advantage to claim. The defensible benefits are transparent certification, local work reduction under structure, and sharper exact precision requirements. Their costs include certificate construction, catalogue storage, enlarged coefficient states, and careful handling of realizability and lifting assumptions.

9 What remains necessary for a full-algorithm improvement

The required next steps differ according to whether the goal is a practical speedup or an improved worst-case bound. Neither follows from the local measurements alone.

9.1 Implement the active recursive regime

A complete implementation must execute the source's actual large-instance construction, packet reduction, finite-field schedule, recursive tensor contractions, and interpolation, not only a deliberately exercised leaf component. It must cover the parameter inequalities that make a reduction active, record which nodes use direct summation, and measure the real dominant cost. The Section-3 symmetry benchmark does not automatically accelerate the sharper Appendix-D algorithm.

9.2 Certify degree, target, and precision at every node

Each recursive node needs a proved polynomial degree or a proved smaller realizable subspace. A target decoder must be valid on that space, and its denominator valuations must be included in the precision plan. Creation-time lifts and inherited precision requirements must remain consistent through every child call. In particular, a local $p + 4$ budget cannot override a stronger precision requirement imposed elsewhere in the recursion.

The cost of finding and certifying a target formula must be charged. Full interpolation may still be required by a parent even if one scalar could be recovered from fewer jets. Deleting decoder channels is not the same as deleting expensive evaluations. Any proposed reduction in the latter requires a new valid information-recovery argument and a comparison with the same required outputs.

9.3 Make symmetry survive the source's constructions

Useful automorphisms must preserve not only the original formula but all relevant measurements, correction rows, factor assignments, node operations, and targets. The current generator finder certifies only a restricted subgroup. A faster finder, compact orbit representation, or safe fallback to A when G has no useful reduction is plausible, but must be separately implemented and measured. No source-wide guarantee of nontrivial symmetry has been established for arbitrary labelled inputs.

9.4 Measure realistic performance and establish a complete cost bound

Practical evidence requires larger and more difficult instances, appropriate established counting or contraction baselines, separate preprocessing and recovery profiles, and end-to-end peak memory. Optimized kernels and more controlled repeated timings would help assess small precision effects. They should not omit the cost of validation that the algorithm itself needs at runtime, even though an external benchmark oracle is appropriately excluded.

For a worst-case improvement, one needs a bit-complexity recurrence including coefficient growth, extension-ring arithmetic, interpolation, precision propagation, orbit discovery, and representative-call cost. The modified recurrence must be solved under the source's full admissible parameter conditions and must improve the claimed bound for the intended input class. A fixed local bit saving or a large speedup on easy symmetric instances does not supply that argument. Full proof-dependency and formal-verification checks are a further, distinct task if one wishes to independently validate the source's complete headline claim.

9.5 Evidence-to-goal checklist

Goal	V4 evidence	Still required
Exact local implementation	Counts, complete jets, cubics, and negative tests agree	Coverage of active recursive constructions
Structured-instance speedup	G outperforms A on eight strict orbit-reduction fixtures	Larger hard cases and symmetry surviving all maps
Lower precision requirement	Certified $p + 4$ cubic and $p + 2$ restricted-family recovery	Valid schedules at all recursive nodes
Lower runtime from precision	Mixed medians; no consistent three-bit advantage	Controlled full-cost evaluation in a relevant regime
Lower memory	G increases peaks; precision changes are small	Compact representations and full-recursion profiling
Better general bound	Not established	Improved rigorous worst-case bit-complexity recurrence

10 Conclusion and supporting files

V4 demonstrates that certified orbit aggregation can accelerate a matched exact finite routine substantially on structured instances, while clearly exposing its memory overhead and failures to accelerate. It also demonstrates exact modular recovery at sharply reduced certified precisions, including a stronger narrow-family decoder, without demonstrating a uniform runtime benefit from those savings. The direct controls are an important corrective: both tested families have simpler target-only solutions, and the precision fixture is already a direct-summation leaf of the full source algorithm.

The next substantive advance would be a valid integration into active recursive nodes with complete cost accounting and realistic baselines. Until then, the results support local exact refinements and transparent verification, not a faster unrestricted SAT algorithm or an independent proof of the source's complete ETH assertion.

This summary draws its tables from the recorded v4 JSON results. It does not rerun the benchmark or alter the earlier reports. The principal support files, relative to the same parent directory, are:

- `eth-benchmark-v4/matched-symmetry/summary.json` and `eth-benchmark-v4/matched-precision/summary.json`: metadata and per-case measurements;
- `eth-benchmark-v4/comparison-v4.json` and `eth-benchmark-v4/comparison-v4.csv`: exact answers and comparisons;
- `eth-benchmark-v4/symmetry-selftest-final.json` and `eth-benchmark-v4/precision-selftest-final.json`: correctness and negative controls;
- `eth-benchmark-v4/source-and-case-sha256.json`, `eth-benchmark-v4/final-verification.json`, and `eth-benchmark-v4/README.txt`: provenance and reproduction details.

Large integers in these files must be read with an arbitrary-precision parser. Converting them to ordinary floating-point spreadsheet cells or JavaScript numbers can destroy exactness.

References

[1] M. Subbarao. *Ram Calculus and the Exponential Time Hypothesis: Exact Finite-Algebra Reformulations, Aggregated Boolean Jets, and Precision-Certified Counting. Revision 4: Matched Symmetry and Degree-Aware Precision Experiments*. October 27, 2026. `Ram-Calculus-Exponential-Time-Hypothesis-v4.tex` and corresponding PDF. Especially Sections 17–18 and Tables 10–17.

[2] M. Subbarao. *ETH V4: Degree-Aware Precision and Certified Symmetry*. Local implementation, raw cases, oracle results, timings, and memory measurements, October 27, 2026. `eth-benchmark-v4/`.

[3] OpenAI. *The Exponential Time Hypothesis is False*. Source manuscript revised October 25, 2026, as pinned and examined in [1]. Repository source: https://github.com/openai/math/tree/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026.